



**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS CONTAGEM - INFORMÁTICA III**

Caio Vitor, Carlos Antonio, Lucas Galarani

RELATÓRIO JETPACK JOYRIDE - LIBGDX

CONTAGEM - MG

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
1.1 Objetivo Geral.....	3
1.2 Objetivos Específicos.....	3
1.3 Tarefas.....	3
1.4 Requisitos Funcionais.....	4
1.5 Requisitos Não Funcionais.....	4
2. DESENVOLVIMENTO.....	5
2.1 Desenvolvimento das Personagens.....	5
2.2 Desenvolvimento dos Inimigos.....	6
2.3 Desenvolvimento dos poderes.....	7
2.4 Desenvolvimento dos Obstáculos.....	7
2.5 Desenvolvimento dos Objetos de Jogo.....	8
2.6 Desenvolvimento do mapa.....	9
2.7 Desenvolvimento dos sons.....	9
3. ESTRUTURAÇÃO DO JOGO E CLASSES.....	10
3.1 Classe Jogo.....	10
3.2 Classe Poder.....	11
3.3 Classe Personagem.....	12
3.4 Classe inimigo.....	12
3.5 Classe Personagem Principal.....	13
3.5 Classe Planos de Fundo.....	13
3.5 Classe Obstáculos.....	14
3.6 Classe GameAssets.....	14
4. CONCLUSÃO.....	15
5.REFERÊNCIAS.....	15

LISTA DE FIGURAS

FIGURA 1 - Sprite Sheet do Personagem Principal (fonte: Manus ai).....	6
FIGURA 2 - Sprite sheet Esqueleto (fonte manus ai).....	7
FIGURA 3 - Sprite sheet Goblin (fonte manus ai).....	7
FIGURA 4 - Sprite sheet Corvo (fonte: manus ai).....	7
FIGURA 5 - Sprite do escudo (fonte: manus ai).....	8
FIGURA 6 - Sprite do ímã (fonte: manus ai).....	8
FIGURA 7 - Sprite do zipper (fonte: Jetpack Joyride fandom).....	9
FIGURA 8 - Sprite do míssil (fonte: criação pessoal).....	9
FIGURA 9 - Sprite da moeda (fonte criação pessoal).....	9
FIGURA 10 - Sprite do coração (fonte criação pessoal).....	10
FIGURA 11 - Classe Jogo (fonte: criação pessoal).....	11
FIGURA 12 - Classe Poder e suas filhas (fonte: criação pessoal).....	12

FIGURA 13 - Classe Personagem (fonte: criação pessoal).....	13
FIGURA 14 - Classe Inimigo (fonte: criação pessoal).....	13
FIGURA 15 - Classe Personagem Principal (fonte: criação pessoal).....	14
FIGURA 16 - Classe Plano de Fundo (fonte: criação pessoal).....	14
FIGURA 17 - Classe Obstáculo(fonte: criação pessoal).....	15
FIGURA 18 - Classe GameAssets(fonte: criação pessoal).....	15

1. INTRODUÇÃO

O presente trabalho tem como objetivo apresentar a criação de um jogo feito através da ferramenta LIBGDX na linguagem Java. O jogo sorteado para o grupo se trata de Jetpack Joyride, um jogo 2D procedural e ao estilo de *endless runner* (corredor infinito).

Através de técnicas de programação de OOP e funcional é feito o jogo completo. A ferramenta utilizada permite tratar sobre tudo que o desenvolvimento de games necessita, mas sem abdicar das técnicas básicas.

1.1 Objetivo Geral

Este trabalho tem como objetivo geral desenvolver um jogo digital 2D com rolagem de cenário utilizando LibGDX, aplicando conceitos de POO e arquitetura de software.

1.2 Objetivos Específicos

- Implementar jogo funcional
- Aplicar POO
- Criar animações e colisões
- Estruturar código modular
- Utilizar Git/GitHub
- Produzir um relatório técnico em conformidade com as normas da ABNT.

Acima se mostra todos os objetivos específicos requisitados para a conclusão do trabalho. Ao analisar o código e o funcionamento do jogo percebe-se que tudo que foi requisitado está completo e em funcionamento.

1.3 Tarefas

O jogo a ser desenvolvido apresenta uma estrutura baseada em entidades, onde cada elemento (jogador, inimigos, obstáculos, cenários e itens) deve ser modelado como uma classe específica dentro do sistema. Essas entidades possuem características próprias e comportamentos distintos, formando uma organização orientada a objetos adequada ao contexto de jogos digitais.

Os estudantes deverão projetar uma classe-base (por exemplo, Entidade ou ObjetoDeJogo) e criar subclasses especializadas (como Jogador, Inimigo, Projétil, Cenário), incorporando atributos e métodos específicos para cada tipo, garantindo reutilização de código e aplicação de herança e polimorfismo.

A aplicação deve implementar completamente o ciclo de execução de um jogo digital (game loop), contemplando atualização de estados, renderização dos elementos na tela, movimentação dos objetos e tratamento de eventos de entrada do usuário.

Deve ser implementado o sistema de rolagem (scrolling), responsável por movimentar o cenário continuamente, além de sistemas de colisão, pontuação e progressão de dificuldade (fases ou aumento gradual de desafio).

Os objetos dinâmicos do jogo (como inimigos e itens) devem ser organizados utilizando estruturas de dados das Collections do Java (listas), permitindo inserção, remoção e atualização eficiente durante a execução.

Analisando as tarefas pedidas, vemos que todas foram concluídas na aplicação e funcionam sem erros no terminal e ao longo do jogo.

1.4 Requisitos Funcionais

- RF01 – Inicialização do jogo
- RF02 – Implementação do jogador
- RF03 – Sistema de rolagem (scrolling)
- RF04 – Interação com objetos
- RF05 – Sistema de colisão
- RF06 – Sistema de pontuação ou progresso
- RF07 – Múltiplas fases ou dificuldade progressiva
- RF08 – Uso de listas (Collections)
- RF09 – Uso de animações (sprites)
- RF10 – Arquitetura orientada a objetos
- RF11 – Utilização de GitHub

Acerca dos requisitos funcionais o jogo também cumpre com todos eles. Uma observação se deve ao RF09 que na criação de sprites foi utilizado Inteligência Artificial para a criação das mesmas. Com isso, e criado um melhor visual ao personagem e itens, além de suas animações feitas seguindo o sprite sheet.

1.5 Requisitos Não Funcionais

- RNF01 – Desempenho
- RNF02 – Usabilidade
- RNF03 – Organização de código
- RNF04 – Portabilidade
- RNF05 – Qualidade visual e sonora

Todos os requisitos funcionais também foram concluídos. Em termos de usabilidade buscou-se seguir o mesmo princípio do jogo original.

2. DESENVOLVIMENTO

Neste capítulo pretende-se tratar sobre o desenvolvimento do jogo, bem como as artes escolhidas para itens e personagens, controles do jogo, estruturação de projeto, classes e mecânicas.

2.1 Desenvolvimento das Personagens

O jogo conta com uma classe Principal Personagem e possui classes filhas como PersonagemPrincipal, Inimigos, Corvo, Esqueleto, Goblin. Através dos conceitos de orientação a objetos como polimorfismo para criação dos inimigos, herança para personagens e encapsulamento o trabalho se dispõe de um funcionamento e conceitos bem utilizados.

Acerca dos Personagens sua construção de visual se deu de forma minimalista e atraente aos jogadores para que continuem jogando o jogo por tempo maior. 7

O personagem Principal que herda atributos e variáveis da classe abstrata Personagem foi desenvolvido com uso de física para os movimentos e getters, funções para o acesso durante o jogo e de outras classes. O visual do personagem como abaixo se mostra simples e cumpre com a inspiração base do jogo original.

Como movimentos do personagem ele pode correr (não exige nenhuma tecla específica) e voar com sua jetpack (apertando a tecla espaço). Seguindo o jogo original, há a mecânica de tiro que é ativada pela tecla G do teclado do usuário.

O personagem ao longo do ciclo de jogo principal pode atirar, obter poderes, correr, pular e morrer. Os poderes são o escudo e o imã que dão ao jogador uma leve ajuda durante o mapa mas sem tirar os aspectos de dificuldade do jogo



FIGURA 1 - Sprite Sheet do Personagem Principal (fonte: Manus ai)

2.2 Desenvolvimento dos Inimigos

Além da classe Personagem, temos a classe Inimigo que herda de Personagem e tem suas classes filhas que são Goblin, Esqueleto e Corvo. Na classe dos inimigos temos na base seus atributos e funções que serão herdados e apenas mudados quando necessários em suas filhas como os movimentos de cada inimigo.

Abordando cada inimigo temos primeiramente o Esqueleto que sua função por padrão é percorrer o mapa da direita para esquerda e caso o jogador encoste nele, tome dano que é algo padrão definido já na classe pai.



FIGURA 2 - Sprite sheet Esqueleto (fonte manus ai)

Juntamente temos o Goblin que também possui o movimento do esquerdo de andar no mapa da direita para a esquerda, só que algo específico de sua classe é poder pular como na sequência de animação que se observa na imagem abaixo. Assim como no esqueleto, o goblin também possui spawn aleatório, porém, no eixo X e com eixo Y já definido para acompanhar o chão



FIGURA 3 - Sprite sheet Goblin (fonte manus ai)

Por último, temos o Corvo que possui o seu voo importado do movimento básico da classe. O corvo possui geração aleatória no eixo Y do jogo e prossegue na tela até sair dela e ser excluído.



FIGURA 4 - Sprite sheet Corvo (fonte: manus ai)

2.3 Desenvolvimento dos poderes

Durante o ciclo de jogo dois poderes/itens podem aparecer. O escudo e o ímã são power-ups que ajudam o jogador durante o ciclo de jogo. É importante ressaltar que esses poderes não deixam o jogador muito forte e não influenciam tanto na jogatina, servindo apenas de uma ajuda como no escudo.

Com uma classe base Poder, o escudo protege o personagem em um tempo de 10s ou então até tomar um dano dentro desse intervalo. Como nos inimigos o escudo possui spawn aleatório e pega-lo mais de uma vez no jogo apenas reseta o tempo da proteção e não cria uma a mais.



FIGURA 5 - Sprite do escudo (fonte: manus ai)

Juntamente, o ímã é responsável por coletar a moeda do mapa durante os 10s de efeito do poder definido em sua classe pai e que possui alcance máximo de 250m dentro do jogo. Assim como o escudo, possui spawn durante o mapa aleatório e também não é acumulativo.



FIGURA 6 - Sprite do ímã (fonte: manus ai)

2.4 Desenvolvimento dos Obstáculos

Ao longo do jogo e aos personagens do jogo foram criados alguns obstáculos que deixam o jogo mais desafiador para quem está jogando. Os obstáculos possuem spawn aleatórios ao

longo do jogo bem como seus sprites e animações e efeitos negativos ao jogador. Alguns deles, como o zapper e o míssil, implementam a interface Explodível, utilizada para determinar se um objeto deve ou não explodir, e se sim, onde.

Ainda sobre essa interface, a explosão, que acontece apenas em objetos de classes explodidos, recebe uma animação que foi implementada de uma maneira diferente em relação às demais: os frames são carregados em um array, que é percorrido quando uma explosão deve acontecer, por meio de um loop que contém um objeto de uma classe estática aninhada à Jogo.java, que guarda o estado individual de uma explosão específica, incluindo seu tempo de vida, que serve como cronômetro, para saber qual frame usar no momento, que então é mostrado dentro da função render(), de Jogo.java.

Um dos elementos principais do jogo e que foi trago do jogo base são os zapper, funcionam como um obstáculo que se move e enquanto isso gira em torno de si mesmo. Eles também dão dano ao personagem assim como os inimigos e seu dano é cancelado pelo escudo caso ativo.



FIGURA 7 - Sprite do zapper (fonte: Jetpack Joyride fandom)

Outro elemento presente no jogo são os mísseis que também possuem spawn aleatório como os zappers ao longo do eixo Y do jogo, e têm algo único, que o escudo não cancela o efeito de dano ao personagem do foguete. Caso o personagem colida com o míssil, ele perde e vai para a tela de morte no exato momento, sendo ele e o zapper os únicos objetos a tirar as três vidas do jogador.



FIGURA 8 - Sprite do míssil (fonte: criação pessoal)

2.5 Desenvolvimento dos Objetos de Jogo

Durante a gameplay temos a presença de moedas, que servem como métrica de habilidade do jogador e que serviriam para comprar power-ups e outras coisas. As moedas não contam com uma função específica além de métrica já que o tempo do trabalho é curto e não foi algo explícito nos requisitos ou objetivos. Basicamente, os planos seriam de haver uma loja assim como no jogo original para o jogador comprar mais coisas que poderiam facilitar sua jogatina. Porém, isso não pode acontecer e as moedas são apagadas após o fechamento ou morte do jogador.

As moedas possuem efeito acumulativo e contam com spawn aleatório ao longo da tela com um eixo Y fixo e sempre em grupos de 3 ou mais moedas.



FIGURA 9 - Sprite da moeda (fonte criação pessoal)

O jogador durante o jogo conta com três vidas. Essas vidas a mais, que não existem no jogo original servem para tornar a experiência de jogo mais otimizada e divertida, evitando ser que reiniciar a sua gameplay toda hora quando morrer. Elas não podem ser obtidas durante o ciclo e só podem se manter caso o personagem não colida com algum inimigo ou esteja com o escudo ativado que, como citado, protege o jogador.



FIGURA 10 - Sprite do coração (fonte criação pessoal)

2.6 Desenvolvimento do mapa

Para a criação do mapa, decidimos implementar duas principais classes: GeradorCenario e GeradorFundo. A primeira é responsável pela geração em scroll do mapa, ela é quem decide qual o próximo bloco de acordo com o bioma e quando os personagens e os objetos de jogo são gerados. A segunda é quem faz as trocas dos planos de fundo no jogo, respeitando os biomas, que são: neve, fogo, concreto e floresta. Cada bioma possui seu chão e fundo específico. Além disso, essas classes são chamadas no arquivo Jogo.java, que organiza quando cada método deve ser chamado, quando algo deve ou não ser executado. A utilização de classes gerenciadoras se deu pelo benefício que elas permitem de facilitar a utilização de object pooling, técnica utilizada para a otimização do jogo, que consiste na reutilização dos objetos criados, ativando-os ou desativando-os quando conveniente.

Outrossim, utilizamos a interface Tile.java, para auxiliar na criação de objetos e, principalmente, dos chãos do mapa. Ela obriga a definição se o objeto tem colisão com o jogador, se ele dá dano ao jogador, e também um getter e um setter da posição do objeto, o que ajudou bastante na codificação não somente do mapa, mas dos objetos de uma forma geral.

2.7 Desenvolvimento dos sons

Todas as músicas do jogo foram criadas pelo grupo, com uma guitarra e pela edição de áudio no Audacity, e implementadas no jogo em seus respectivos lugares – a música de menu toca no menu apenas, a de morte na tela de morte, e a música 1 e 2 tocam ao iniciar o jogo ou ao a música que está tocando acabar, cada uma com 50% de chance de ser tocada ao iniciar o jogo. Outros sons do jogo foram extraídos da internet e tratados no mesmo software, para se adequarem melhor ao nosso programa.

3. ESTRUTURAÇÃO DO JOGO E CLASSES

O jogo conta ao todo com 35 classes e interfaces para o seu funcionamento. Dentre elas temos classes abstratas, filhas, polimórficas e etc.

3.1 Classe Jogo

A principal classe do jogo e onde roda toda a aplicação é a classe Jogo.java que juntamente com a ManagerAssets.java é responsável por gerenciar tudo da aplicação que é entregue a ela. Nela se observa a aplicação de diversos conceitos aprendidos em sala e seu diagrama de classes está abaixo:



FIGURA 11 - Classe Jogo (fonte: criação pessoal)

3.2 Classe Poder

Outra classe que possui papel fundamental e é gerenciada por Jogo é a classe Poder, nela são criados o imã e o escudo, poderes que estarão ao longo do jogo inteiro e também possui polimorfismo em sua aplicação para gerenciamento por ArrayList dos poderes ao longo do ciclo de jogo. Ela pode ser observada abaixo juntamente com suas filhas:

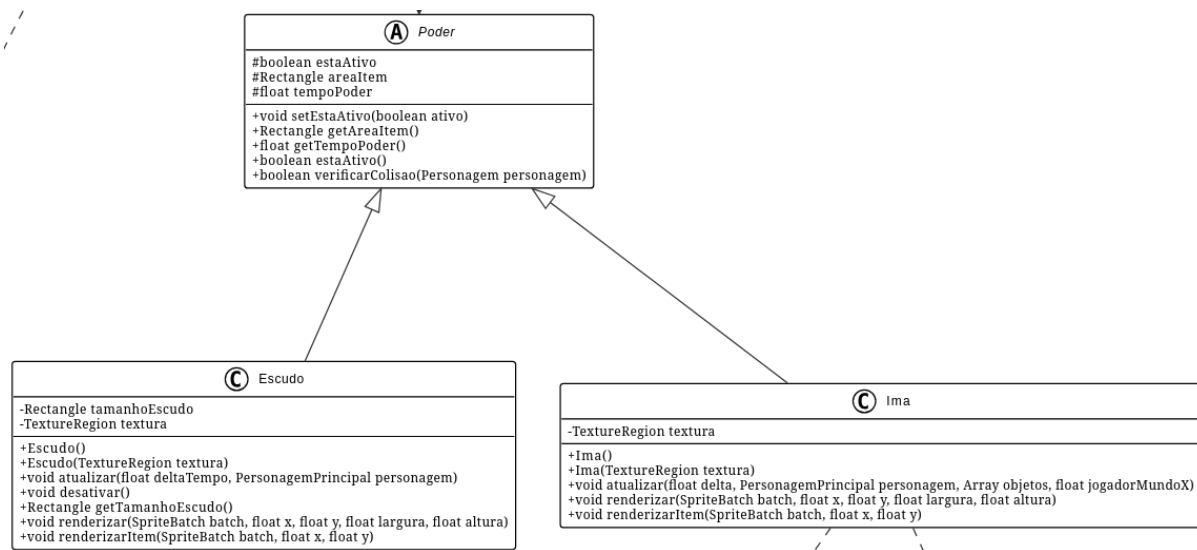


FIGURA 12 - Classe Poder e suas filhas (fonte: criação pessoal)

3.3 Classe Personagem

Esta classe abstrata gerencia todos os inimigos e o personagem no qual o jogador controla. A classe personagem também contém a ideia de herança como o restante das coisas mas não possui polimorfismo diretamente a ela, tendo somente em sua filha Inimigo. Pode-se ver ela abaixo:

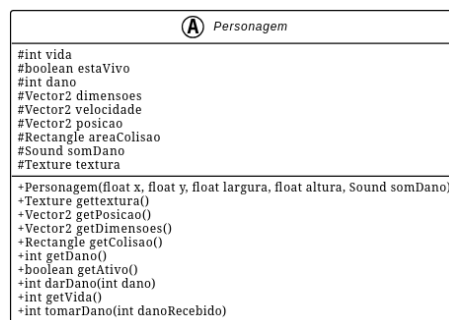


FIGURA 13 - Classe Personagem (fonte: criação pessoal)

3.4 Classe inimigo

Esta classe abstrata é responsável especificamente pelo gerenciamento de cada inimigo do jogo, nela são definidos atributos e métodos básicos de todos os inimigos que por sua vez são herdados pelos seus filhos que de fato são os inimigos que vemos na tela, cada um com seus atributos e métodos em específico.

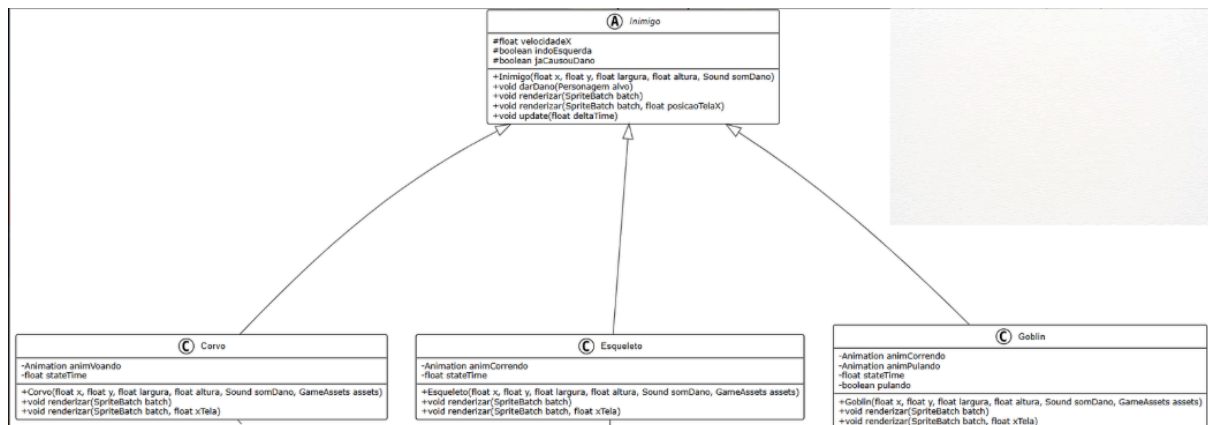


FIGURA 14 - Classe Inimigo (fonte: criação pessoal)

3.5 Classe Personagem Principal

Classe que armazena e administra todos os métodos e atributos relacionados ao personagem principal, armazena os estados do mesmo como se ele está vivo ou morto, além de ser responsável por implementar a sua movimentação.

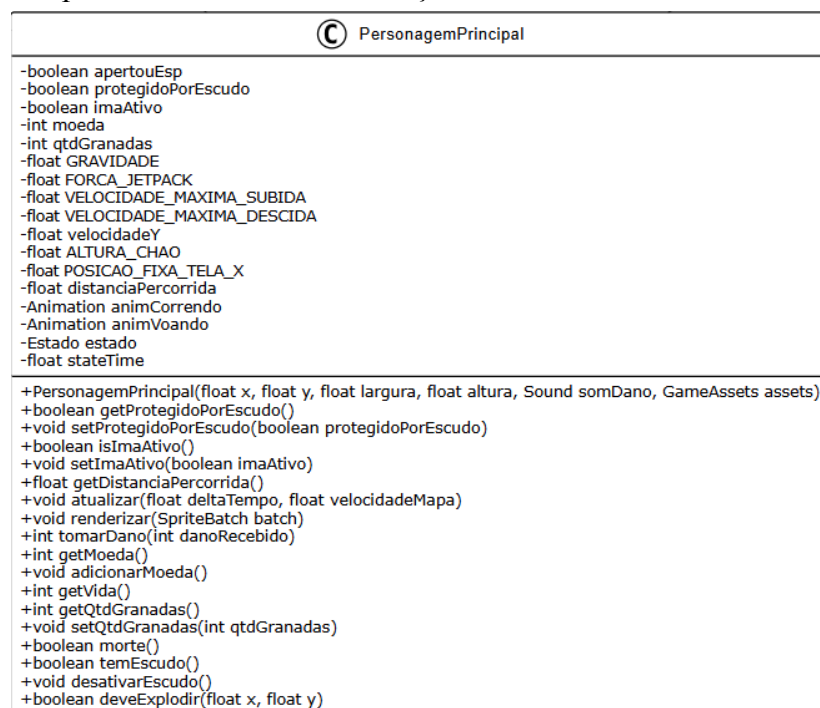


FIGURA 15 - Classe Personagem Principal (fonte: criação pessoal)

3.5 Classe Planos de Fundo

Esta classe abstrata define e armazena os atributos relacionados ao fundo do jogo, todos os fundos do jogo (fundo de fogo, neve, concreto e floresta) a tem como classe pai.

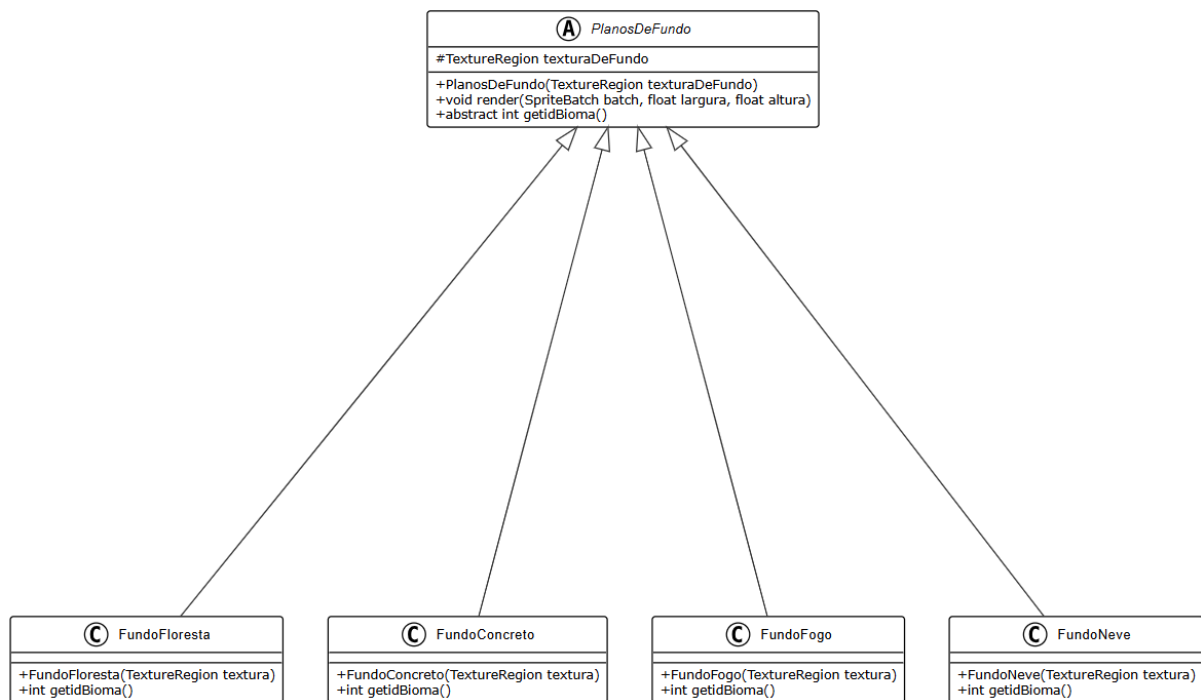


FIGURA 16 - Classe Plano de Fundo (fonte: criação pessoal)

3.5 Classe Obstáculos

Classe abstrata que contém todos os obstáculos do jogo, a mesma tem como filha as classes que representam os obstáculos em si, sendo eles o míssil e o zapper.

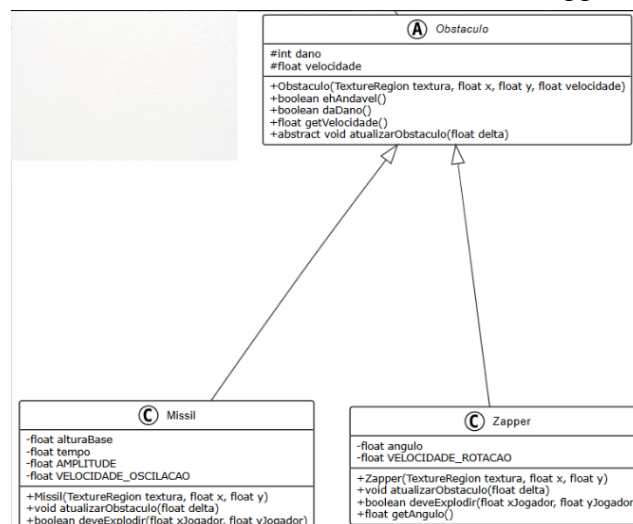


FIGURA 17 - Classe Obstaculo(fonte: criação pessoal)

3.6 Classe GameAssets

A classe **GameAssets** funciona como o almoxarifado central do jogo, sendo a única responsável por carregar, organizar e liberar todos os recursos visuais e sonoros da aplicação

por meio do `AssetManager` da `LibGDX`. Além de centralizar o carregamento de músicas, efeitos sonoros e texturas de cenários, ela desempenha um papel fundamental na criação das animações, fatiando as folhas de sprites (*spritesheets*) do personagem e dos inimigos em matrizes de quadros (`TextureRegion`) prontas para a execução. Ao aplicar filtros de renderização para otimizar os gráficos e garantir o descarte adequado desses arquivos da memória ao encerrar o jogo, a classe assegura tanto a fluidez visual quanto o desempenho técnico do sistema.



FIGURA 18 - Classe GameAssets(fonte: criação pessoal)

4. CONCLUSÃO

Diante do desenvolvimento deste projeto, conclui-se que o objetivo de reproduzir as dinâmicas e o funcionamento do clássico *Jetpack Joyride* foi alcançado com sucesso. O processo de desenvolvimento utilizando a linguagem Java e o framework `LibGDX` serviu como um excelente laboratório prático para consolidar os conceitos teóricos de Programação Orientada a Objetos vistos em sala de aula, tais como herança, polimorfismo, classes abstratas e encapsulamento. A arquitetura adotada, dividida em entidades modulares e centralizada por um forte controle de ciclo de vida e gerenciamento de memória (através do *Asset Manager*), provou ser eficiente para lidar com elementos dinâmicos, incluindo o fatiamento de *spritesheets*, tratamento de colisões e controle do fluxo do jogo.

Pretendia-se caso houvesse maior tempo implementar mais coisas ao jogo e semelhanças ao jogo original como uma loja, já que as moedas não possuem nenhuma função como dito, criar mais obstáculos e inimigos, além de aperfeiçoar as animações dos personagens existentes.

Por fim, analisando tudo que foi proposto ao trabalho seja por parte de relatório ou por programação do jogo, tudo foi cumprido sem ressalvas e com êxito. O jogo é entregue em perfeito estado e com nenhum sinal de alerta.

5. REFERÊNCIAS

8-BIT Coin Sound Effect (Copyright Free). YouTube, 2021. 1 vídeo (11 s). Publicado pelo canal Sound Effects. Disponível em: <https://www.youtube.com/watch?v=5v20ztxGvQ0>. Acesso em: 14 jun. 2026.

AUDACITY. **Audacity.** Versão 3.x. [S. l.]: Muse Group, 2026. Software. Disponível em: <https://www.audacityteam.org/>. Acesso em: 10 jun. 2026.

CREATOR ASSETS. **8-Bit Powerup Sound Effects (Copyright Free).** YouTube, 2021. 1 vídeo (48 s). Disponível em: <https://www.youtube.com/watch?v=SoZhpnTuQBo>. Acesso em: 14 jun. 2026.

DATA CAMP. **Polimorfismo em Java: o que é e como funciona.** 2026. Disponível em: <https://www.datacamp.com/pt/doc/java/polymorphism>. Acesso em: 5 jun. 2026.

DESCOTTES, Julien. **Piskel: free online sprite editor.** Versão 0.14.0. [S. l.], 2026. Software. Disponível em: <https://www.piskelapp.com/>. Acesso em: 8 jun. 2026.

DEV MEDIA. **Encapsulamento, polimorfismo e herança em Java.** 2009. Disponível em: <https://www.devmedia.com.br/encapsulamento-polimorfismo-heranca-em-java/12991>. Acesso em: 3 jun. 2026.

DEV MEDIA. **Uso de polimorfismo em Java.** 2012. Disponível em: <https://www.devmedia.com.br/uso-de-polimorfismo-em-java/26140>. Acesso em: 3 jun. 2026.

Difference between LibGDX and LWJGL. Game Development Stack Exchange, 20 jul. 2016. Disponível em: <https://gamedev.stackexchange.com/questions/125820/difference-between-libgdx-and-lwjgl>. Acesso em: 29 jun. 2026.

HOUDHARY, Awais. **Jetpack-Joyride: remake of Jetpack Joyride game.** GitHub, 2024. Disponível em: <https://github.com/awaischoudhary/Jetpack-Joyride>. Acesso em: 29 jun. 2026.

Jetpack Joyride. In: WIKIPÉDIA: a enciclopédia livre. 2026. Disponível em: https://en.wikipedia.org/wiki/Jetpack_Joyride. Acesso em: 29 jun. 2026.

LIBGDX. **Creating a project.** 2026. Disponível em: <https://libgdx.com/wiki/start/project-generation>. Acesso em: 29 jun. 2026.

LIBGDX. **Demos & tutorials.** 2026. Disponível em: <https://libgdx.com/wiki/start/demos-and-tutorials>. Acesso em: 29 jun. 2026.

LIBGDX. **External tutorials.** 2026. Disponível em: <https://libgdx.com/wiki/articles/external-tutorials>. Acesso em: 29 jun. 2026.

LIBGDX. **Features.** 2026. Disponível em: <https://libgdx.com/features/>. Acesso em: 29 jun. 2026.

LIBGDX. **LibGDX: desktop/Android/HTML5/iOS Java game development framework.** GitHub, 2026. Disponível em: <https://github.com/libgdx/libgdx>. Acesso em: 29 jun. 2026.

LIBGDX. **LibGDX.** 2026. Disponível em: <https://libgdx.com>. Acesso em: 29 jun. 2026.

LIBGDX. **Modules overview.** 2026. Disponível em: <https://libgdx.com/wiki/app/modules-overview>. Acesso em: 29 jun. 2026.

LIBGDX. **Showcase.** 2026. Disponível em: <https://libgdx.com/showcase/>. Acesso em: 29 jun. 2026.

LIBGDX. **Status reports.** 2026. Disponível em: <https://libgdx.com/news/devlog/>. Acesso em: 29 jun. 2026.

LIBGDX. **Wiki.** 2026. Disponível em: <https://libgdx.com/wiki/>. Acesso em: 29 jun. 2026.

LibGDX. In: WIKIPÉDIA: a enciclopédia livre. 2026. Disponível em: <https://en.wikipedia.org/wiki/LibGDX>. Acesso em: 29 jun. 2026.

LibGDX: the open-source cross-platform game development framework. GitHub, 2026. Disponível em: <https://libgdx.com>. Acesso em: 1 jun. 2026.

MAGNIFIC. **Cartoon laboratory room with machine.** 2026. Disponível em: https://www.magnific.com/free-vector/cartoon-laboratory-room-with-machine_13176166.htm. Acesso em: 12 jun. 2026.

MAGNIFIC. **Infernal dark cave with lava scene.** 2026. Disponível em: https://www.magnific.com/free-vector/infernal-dark-cave-with-lava-scene_12321258.htm. Acesso em: 12 jun. 2026.

MAGNIFIC. **Pixel art rural landscape background.** 2026. Disponível em: https://www.magnific.com/free-vector/pixel-art-rural-landscape-background_49685499.htm. Acesso em: 12 jun. 2026.

MAGNIFIC. **Vector seamless night forest background illustration with starry sky horizontally repeatable.** 2026. Disponível em: https://www.magnific.com/free-vector/vector-seamless-night-forest-background-illustration-with-starry-sky-horizontally-repeatable_132099795.htm. Acesso em: 12 jun. 2026.

MEEGLE. **Game engine for LibGDX.** 5 fev. 2026. Disponível em: https://www.meegle.com/en_us/topics/game-engine/game-engine-for-libgdx. Acesso em: 29 jun. 2026.

MRHOWTOMAKEEVERTHING. **Missile fly sound effect.** YouTube, 2016. 1 vídeo (4 s). Disponível em: <https://www.youtube.com/watch?v=kiOimRGITxI>. Acesso em: 14 jun. 2026.

REDDIT. **Future of LibGDX, and how could we improve it?** Reddit, 2026. Disponível em: https://www.reddit.com/r/libgdx/comments/1thgrnq/future_of_libgdx_and_how_could_we_improve_it/. Acesso em: 29 jun. 2026.

REDDIT. **Slay the Spire is made with LibGDX.** Reddit, 2018. Disponível em: https://www.reddit.com/r/libgdx/comments/7slka6/slay_the_spire_is_made_with_libgdx/. Acesso em: 29 jun. 2026.

SANDU, Bogdan. **The Best Java Game Engines.** TMS Outsource, 20 ago. 2025. Disponível em: <https://tms-outsource.com/blog/posts/java-game-engines/>. Acesso em: 29 jun. 2026.

SOUND EFFECTS - NO COPYRIGHT & FREE DOWNLOAD. **8 Bit Explosion Long Downsampling Sound Effect - Free Download & No Copyright.** YouTube, 2020. 1 vídeo (3 s). Disponível em: <https://www.youtube.com/watch?v=6xoh1VrM8Hc>. Acesso em: 14 jun. 2026.

Technical Evolution and Commercial Landscape of Java-Based Game Development: A Comprehensive Analysis of libGDX, the JVM Ecosystem, and the Jetpack Joyride Model. [S. l.], 2026.

VKY SOUNDS. **8 Bit-Hit: Free Sound Effect [HD].** YouTube, 2015. 1 vídeo (2 s). Disponível em: https://www.youtube.com/watch?v=xxO0hjt7_60. Acesso em: 14 jun. 2026.